

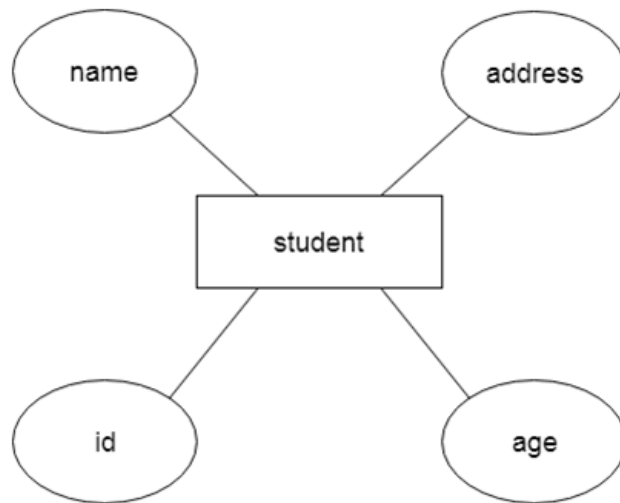
Unit -3

Introduction to Database System

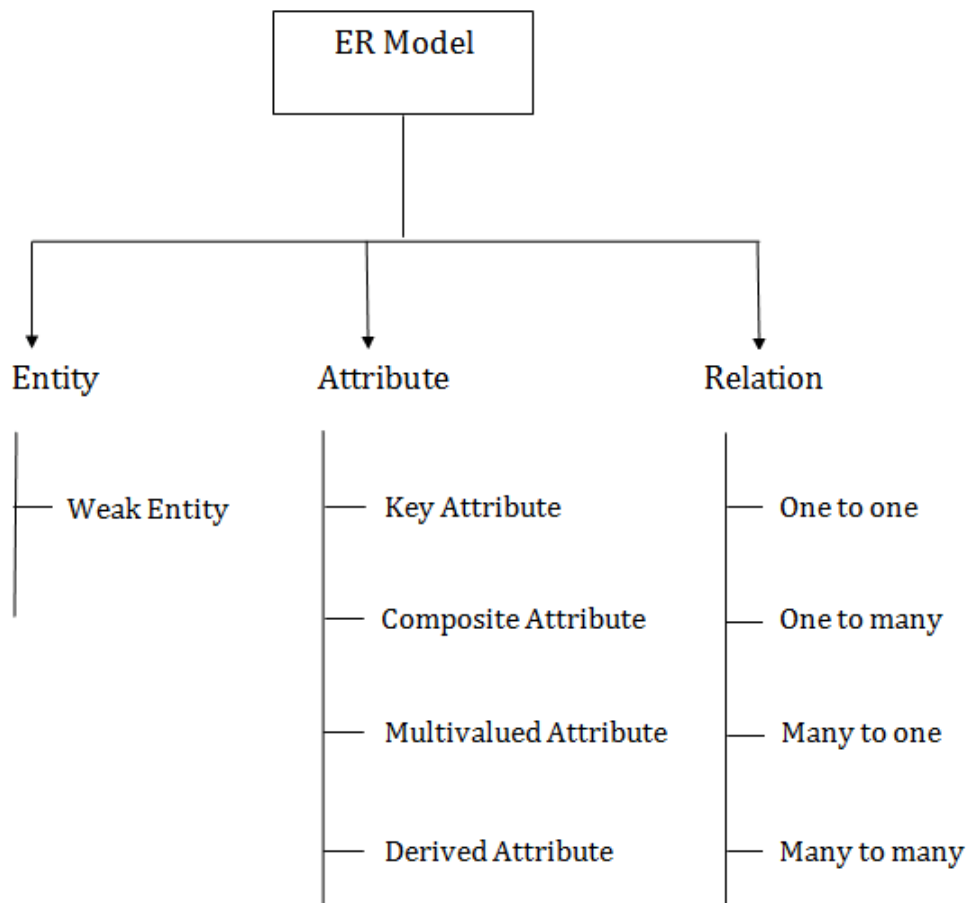
ER (Entity Relationship) Diagram

- ER model stands for an Entity-Relationship model. This model is used to define the data elements and relationship for a specified system.
- It develops a conceptual design for the database. It also develops a very simple and easy to design view of data.
- In ER modeling, the database structure is portrayed as a diagram called an entity-relationship diagram.
- Used to represent real world scenarios.

For example, Suppose we design a school database. In this database, the student will be an entity with attributes like address, name, id, age, etc. The address can be another entity with attributes like city, street name, pin code, etc and there will be a relationship between them.



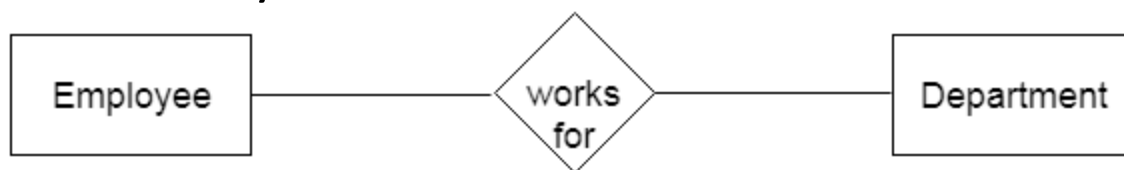
Component of ER Diagram



1. Entity:

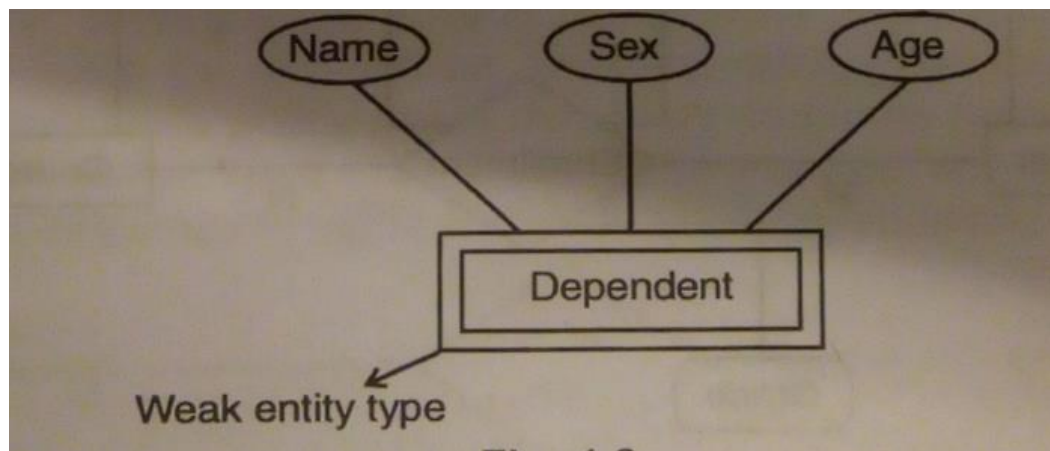
An entity may be any object, class, person or place. In the ER diagram, an entity can be represented as rectangles.

Consider an organization as an example- manager, product, employee, department etc. can be taken as an entity.



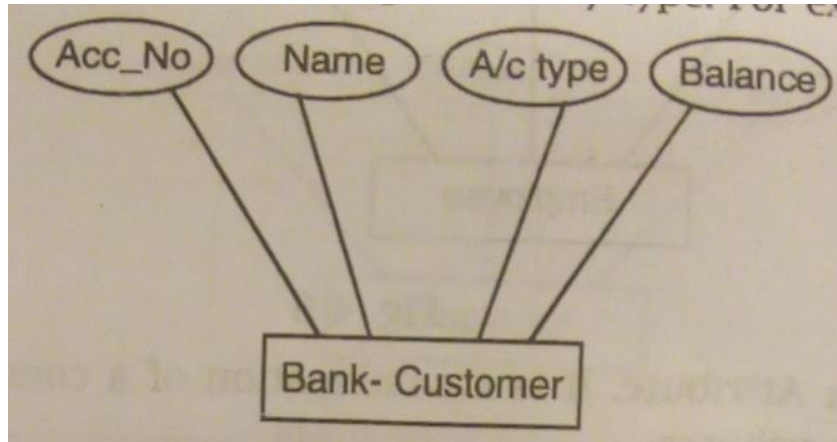
a. Weak Entity

An entity that depends on another entity called a weak entity. The weak entity doesn't contain any key attribute of its own. The weak entity is represented by a double rectangle.



b. Strong Entity Type:-

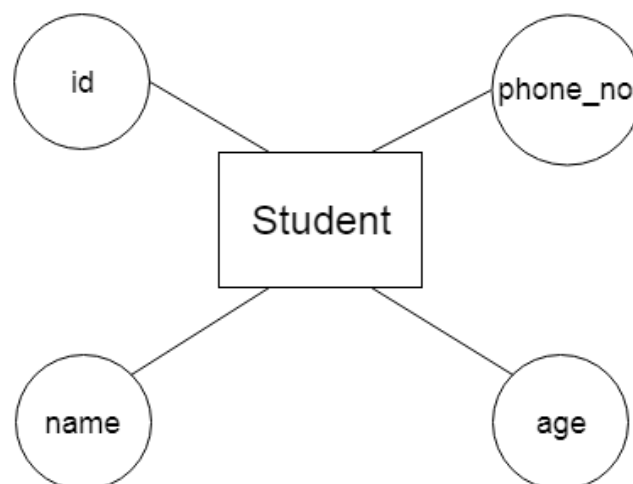
Entity types that have key attribute or primary key are called strong entity type.



2. Attribute

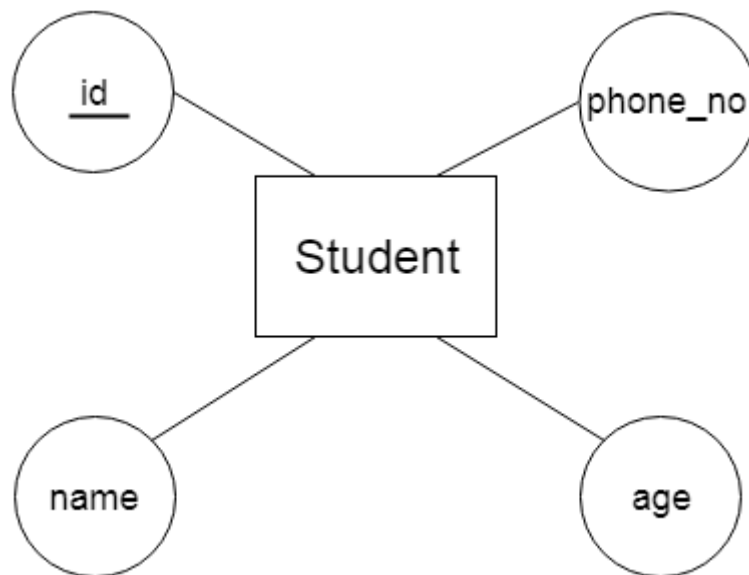
The attribute is used to describe the property of an entity. Eclipse is used to represent an attribute.

For example, id, age, contact number, name, etc. can be attributes of a student.



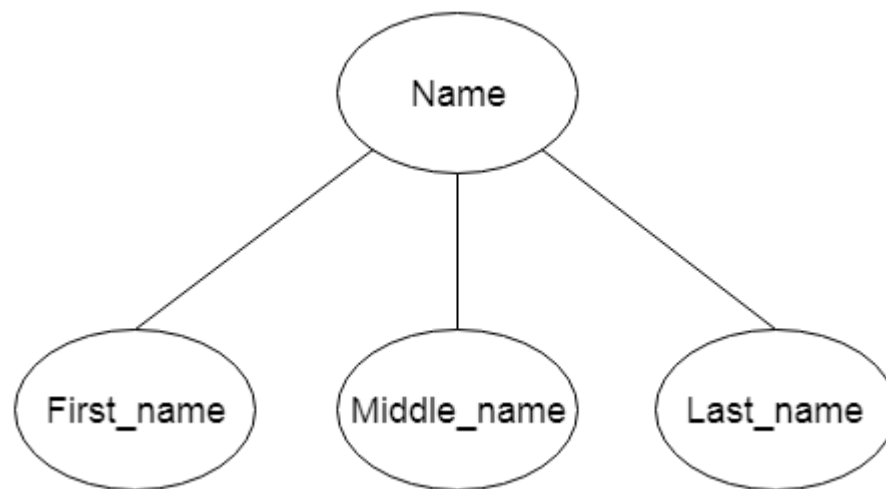
a. Key Attribute

The key attribute is used to represent the main characteristics of an entity. It represents a primary key. The key attribute is represented by an ellipse with the text underlined.



b. Composite Attribute

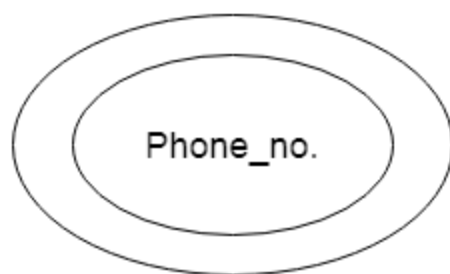
An attribute that composed of many other attributes is known as a composite attribute. The composite attribute is represented by an ellipse, and those ellipses are connected with an ellipse.



c. Multivalued Attribute

An attribute can have more than one value. These attributes are known as a multivalued attribute. The double oval is used to represent multivalued attribute.

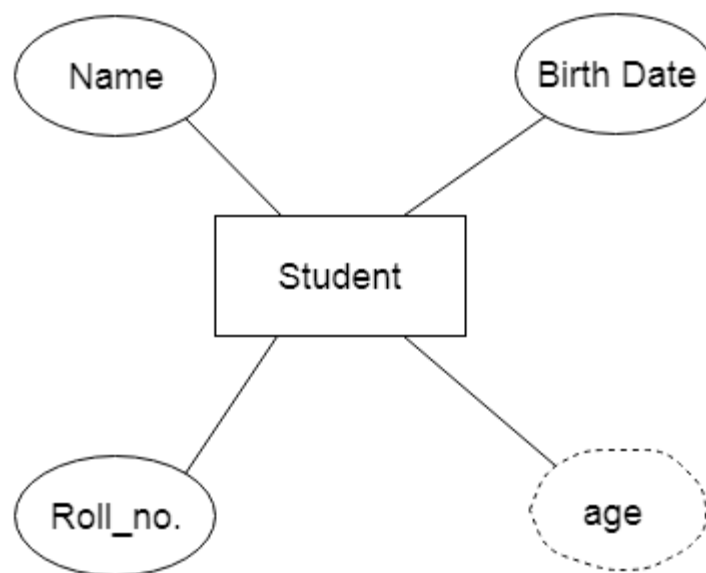
For example, a student can have more than one phone number.



d. Derived Attribute

An attribute that can be derived from other attribute is known as a derived attribute. It can be represented by a dashed ellipse.

For example, A person's age changes over time and can be derived from another attribute like Date of birth.



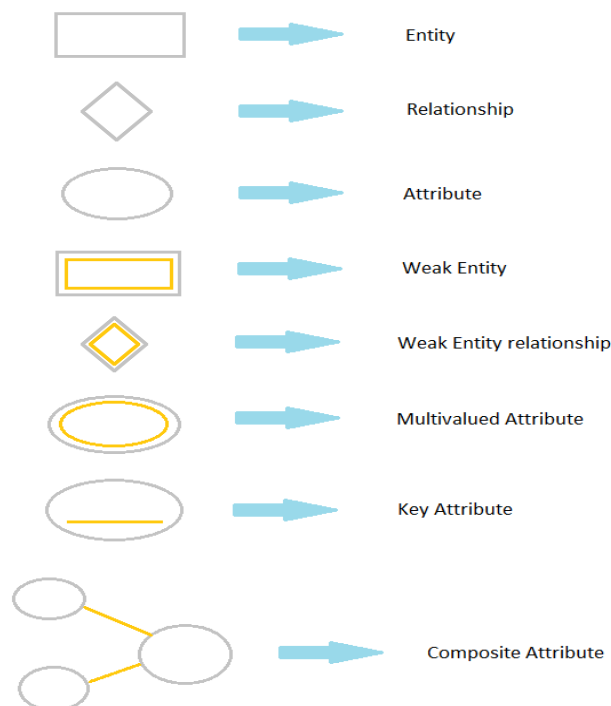
e. Single valued attributes:- An attribute that has only single value for an entity is known as single valued attribute.

For example, assume Student is an entity and its attributes are Name, Age, Address and Phone no. Here the age of student can have only one value. Here, age is single valued attribute.

f.Null Valued Attributes:- An attribute, which has not any value for an entity is known as null valued attribute.

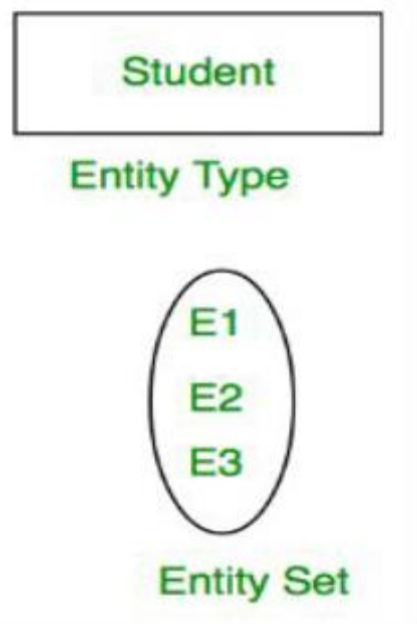
For example, assume Student is an entity and its attributes are Name, Age, Address and Phone no. There may be chance when a student has no phone no. In that case, phone no is called null valued attributes.

ER Diagrams symbols and Notations:-

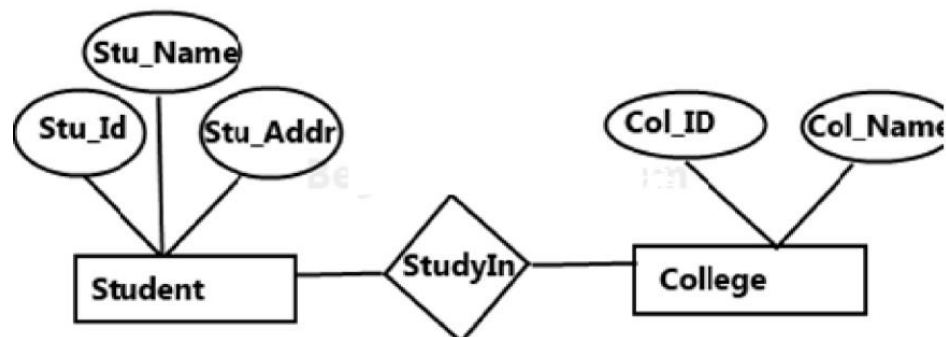


Entity Set:- An entity set is a set of entities of the same type (e.g., all persons having an account at a bank).

An entity type or set is a collection of entities that have the same attributes, or in simple words it is a group of similar object of concern to an organization for which it maintain data.



Example of ER Diagram



Relationship

A relationship is used to describe the relation between entities. Diamond or rhombus is used to represent the relationship.



Types of relationship are as follows:

a. One-to-One Relationship

When only one instance of an entity is associated with the relationship, then it is known as one to one relationship.

For example, A female can marry to one male, and a male can marry to one female.

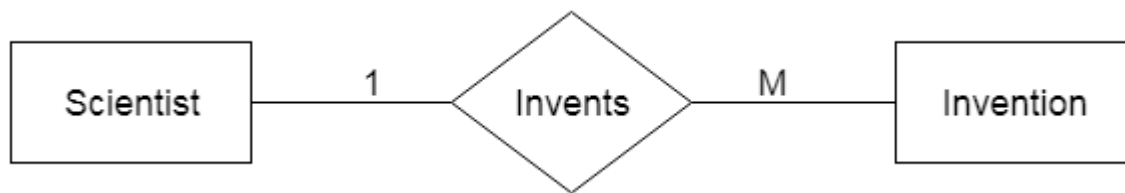


b. One-to-many relationship

When only one instance of the entity on the left, and more than one instance of an entity on the

right associates with the relationship then this is known as a one-to-many relationship.

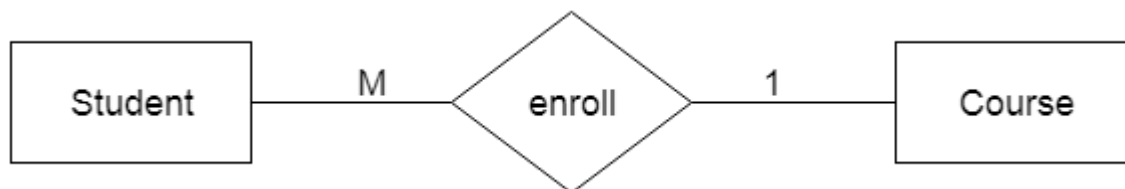
For example, Scientist can invent many inventions, but the invention is done by the only specific scientist.



c. Many-to-one relationship

When more than one instance of the entity on the left, and only one instance of an entity on the right associates with the relationship then it is known as a many-to-one relationship.

For example, Student enrolls for only one course, but a course can have many students.



d. Many-to-many relationship

When more than one instance of the entity on the left, and more than one instance of an entity on the

right associates with the relationship then it is known as a many-to-many relationship.

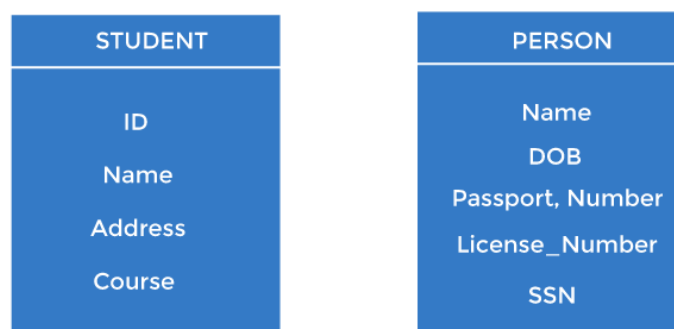
For example, Employee can assign by many projects and project can have many employees.



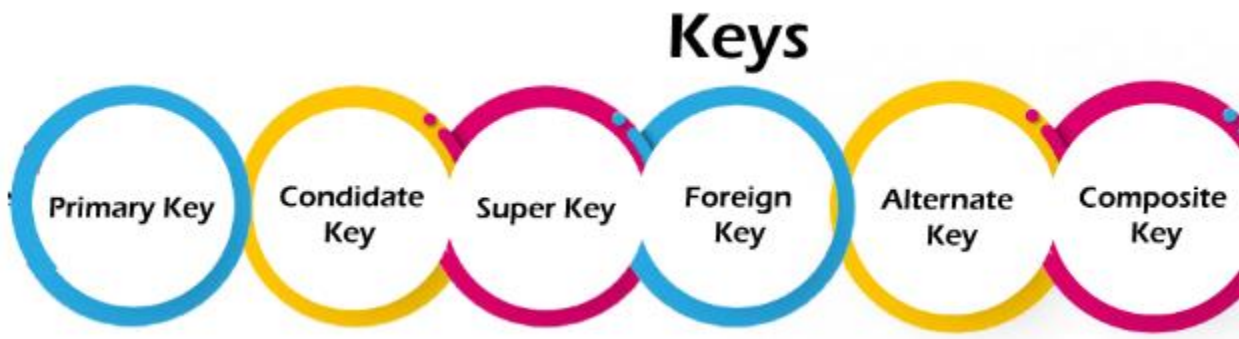
Keys

- Keys play important role in relational database.
- It is used to uniquely identify any record or row of data from table. It is also used to establish and identify relationships between tables.

For example, ID is used as a key in the Student table because it is unique for each student. In the PERSON table, passport_number, license_number, SSN are keys since they are unique for each person.

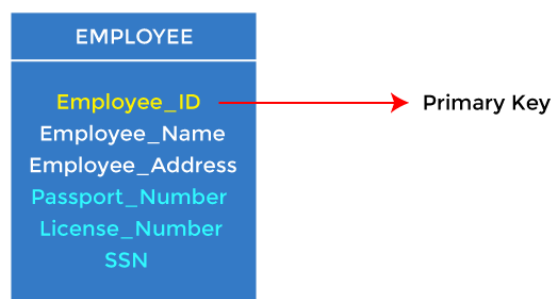


Types of keys:



1. Primary key

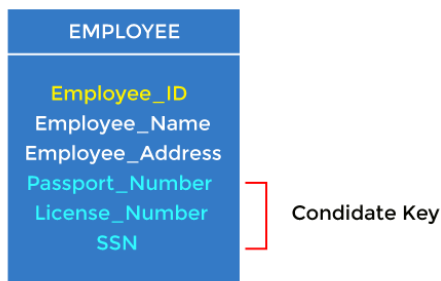
- It is the first key used to identify one and only one instance of an entity uniquely. An entity can contain multiple keys, as we saw in the PERSON table. The key which is most suitable from those lists becomes a primary key.
- In EMPLOYEE table, ID can be primary key since it is unique for each employee. We can even select License_Number and Passport_Number as primary keys since they are also unique.
- The primary key selection is based on requirements and developers.



2. Candidate key

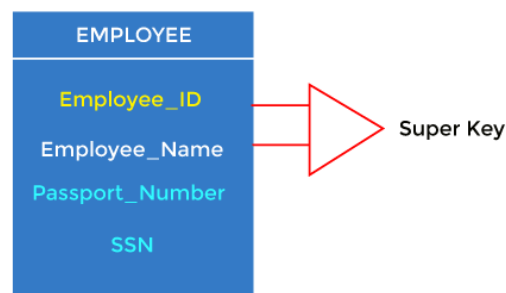
- A candidate key is an attribute or set of attributes that can uniquely identify a tuple.
- Except for the primary key, the remaining attributes are considered a candidate key. The candidate keys are as strong as the primary key.

For example: In EMPLOYEE table, id is best suited for the primary key. The rest of the attributes, like SSN, Passport_Number, License_Number, etc., are considered a candidate key.



3. Super Key

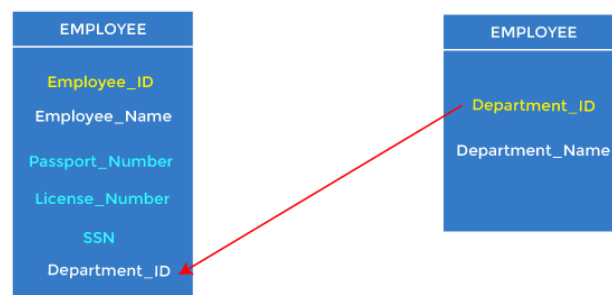
Super key is an attribute set that can uniquely identify a tuple. A super key is a superset of a candidate key.



For example: In EMPLOYEE table, for(EMPLOYEE_ID, EMPLOYEE_NAME), the name of two employees can be the same, but their EMPLOYEE_ID can't be the same. Hence, this combination can also be a key.

4. Foreign key

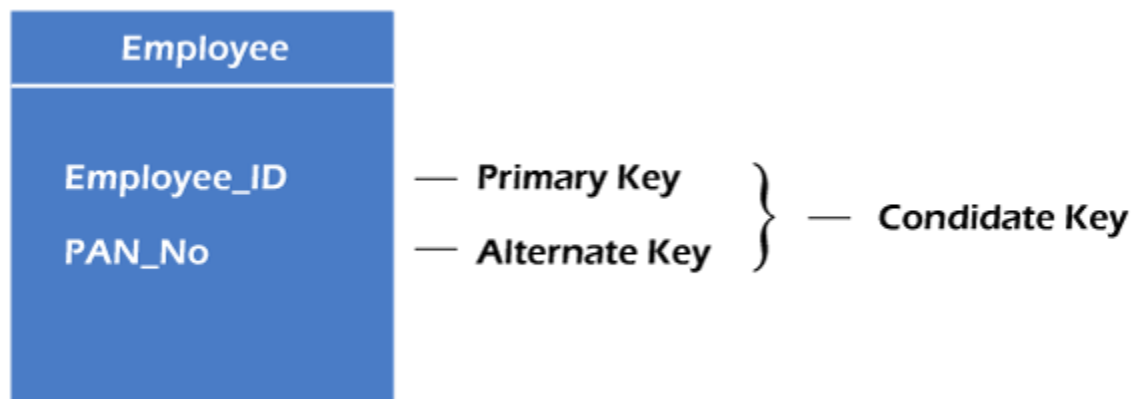
- Foreign keys are the column of the table used to point to the primary key of another table.
- Every employee works in a specific department in a company and employee and department are two different entities. So we can't store department's information in employee table. That's why we link these two tables through primary key of one table.
- We add primary key of DEPARTMENT table, Department_Id, as a new attribute in the EMPLOYEE table.
- In the EMPLOYEE table, Department_Id is the foreign key, and both the tables are related.



5. Alternate key

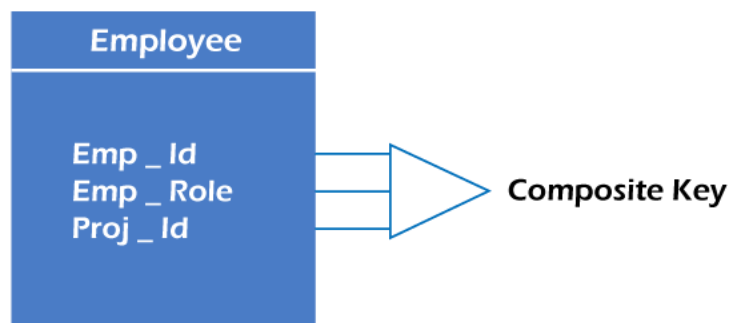
There may be one or more attributes or a combination of attributes that uniquely identify each tuple in a relation. These attributes or combinations of the attributes are called the candidate keys. One key is chosen as the primary key from these candidate keys, and the remaining candidate key, if it exists, is termed the alternate key. In other words, total number of alternate keys is the total number of candidate keys minus primary key. If there is only one candidate key in a relation, it does not have an alternate key.

For example, employee relation has two attributes, Employee_Id and PAN_No, that act as candidate keys. In this relation, Employee_Id is chosen as the primary key, so the other candidate key, PAN_No, acts as the Alternate key.



6. Composite key

Whenever a primary key consists of more than one attribute, it is known as a composite key. This key is also known as Concatenated Key.



For example, in employee relations, we assume that an employee may be assigned multiple roles, and an employee may work on multiple projects. So primary key will be composed of all three attributes, Emp_ID, Emp_role and Proj_ID in combination. So these attributes act as a composite key since the primary key comprises more than one attribute.

Data models

- Data models define how the logical structure of a database is modeled.
- Data models define how data is connected to each other and how they are processed and stored inside the system

- The purpose of a data model is to represent data and to make the data understandable.
- Data Models are fundamental entities to introduce abstraction in a DBMS.

a)Relational Data Model

- In the relational data model, we use tables to represent data and the relationship among that data.
- Each of the tables in the relational data model has a unique name.
- A table has multiple columns where each column name is unique. A table holds records which has value for each column of the table.
- We also refer to the relational database model as a record-based data model as it holds - records of fixed-format.
- The relational database model is the most currently used data model.

Advantages

1. Relational Model are used to manage large amounts of data, such as customer information, employee data, and financial records.

2. It allows users to quickly and easily access data from multiple tables.
3. Data is organized in a structured way, it is easier to update and maintain.
4. Relational Models are most commonly used DBMS and used in wide variety of applications from banking systems to retail websites.
5. Examples of relational DBMS are Oracle, MySQL.

Disadvantages

1. High hardware cost: - In order to separate the physical data information from the logical data, more powerful system hardware – memory is required. This makes the cost of database high.

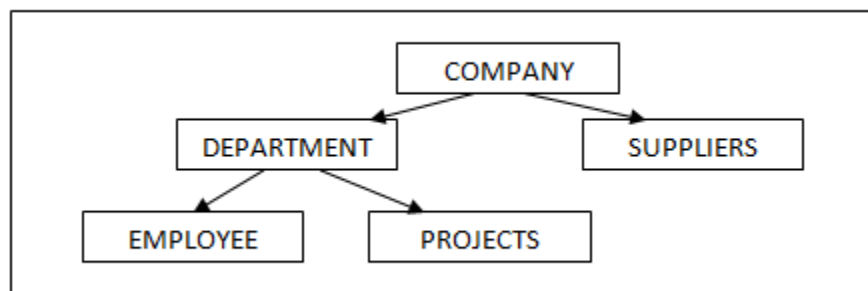
The diagram illustrates a table structure with the following components:

- Table name:** students
- Column name:** RollNo, Name, Phone
- Tuple / Row:** Individual data entries (e.g., 1, Alex, 444123)
- Table / Relation:** The entire set of data (the table itself)
- Attribute / Column:** The individual columns (RollNo, Name, Phone)

RollNo	Name	Phone
1	Alex	444123
2	Aryan	421456
3	Parker	414259
4	Jhon	456785

b) Hierarchical Model

- A hierarchical database model is a data model in which the data is organized into a tree-like structure.
- The data is stored as records which are connected to one another through links.
- A record is a collection of fields, with each field containing only one value.
- The entity type of a record defines which fields the record contains.
- A record in the hierarchical database model corresponds to a row (or tuple) in the relational database model and an entity type corresponds to a table (or relation).
- In order to retrieve data from a hierarchical database the whole tree needs to be traversed starting from the root node.



Advantages

- It helps to address the issues of flat file data storage.
- In flat files, data will be scattered and there will not be any proper structuring of the data.
- This model groups the related data into tables and defines the relationship between the tables, which is not addressed in flat files.

Disadvantage

- It fails to handle many to many relationships efficiently. It results in redundancy and confusion. It can handle only parent-child kind of relationship.
- If we need to fetch any data in this model, we have to start from the root of the model and traverse through its child till we get the result.
- In order to perform traversing, either we should know well in advance the layout of model or we should be very good programmer. Hence fetching through this becomes bit difficult.

c)The Network model

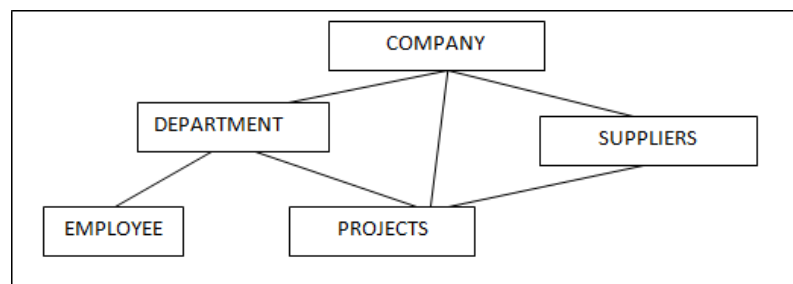
- This is the enhanced version of hierarchical data model. It is designed to address the drawbacks of the hierarchical model.
- This data model is also represented as hierarchical, but this model will not have single parent concept.
- Any child in the tree can have multiple parents
- The Network model replaces the hierarchical tree with a graph thus allowing more general connections among the nodes.
- The main difference of the network model from the hierarchical model, is its ability to handle many to many (N: N) relations.
- Allows a record to have more than one parent.

Advantages

- This model has ability to manage one-to-one (1:1) and many-to-many (N: N) relationships.
- Accessing data is simpler compared to the hierarchical model.
- Data Integrity: In network model, there's always connection between parent and child segments because it depends on parent-child relationship

Disadvantages

- There is no independence between any objects. Hence any changes to object will need change whole model. Hence difficult to manage.
- It would be little difficult to design the relationship between the entities, since all the entities are related in some way.
- System Complexity: Each and every record has to be maintained with the help of pointers, which makes database structure more complex.
- Functional Flaws: Because a great number of pointers is essential, insertion, updates, and deletion become more complex.



Relational Model Concept

- Relational model can represent as a table with columns and rows. Each row is known as tuple. Each table of column has a name or attribute.

- **Domain:** It contains a set of atomic values that an attribute can take.
- **Attribute:** It contains name of a column in table. Each attribute must have a domain.
- **Relational instance:** In relational db, relational instance is represented by a finite set of tuples. Relation instances don't have duplicate tuples.
- **Relational schema:** Relational schema contains name of relation and name of all columns.
- **Relational key:** In the relational key, each row has one or more attributes. It can identify the row in the relation uniquely.

Example: STUDENT Relation

NAME	ROLL_NO	PHONE_NO	ADDRESS	AGE
Ram	14795	7305758992	Noida	24
Shyam	12839	9026288936	Delhi	35
Laxman	33289	8583287182	Gurugram	20
Mahesh	27857	7086819134	Ghaziabad	27
Ganesh	17282	9028 9i3988	Delhi	40

- In the given table, Name, Roll_no, Phone_No, Address, and Age are the attributes.
- The instance of schema Student has 5 tuples.
- t3 = <Laxman, 33289, 8583287182, Gurugram, 20>

Properties of Relations

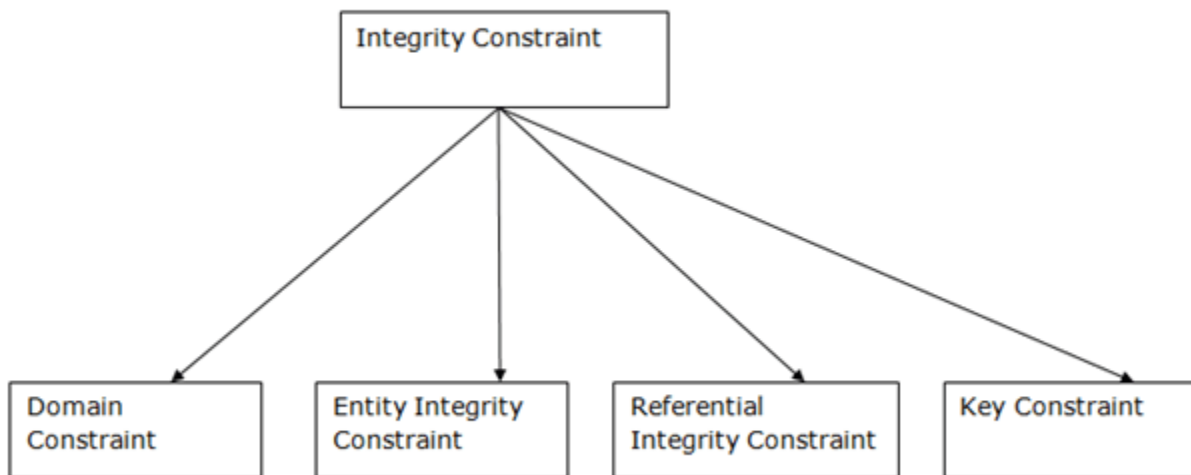
- Name of the relation is distinct from all other relations.
- Each relation cell contains exactly one atomic (single) value
- Each attribute contains a distinct name
- Attribute domain has no significance
- Tuple has no duplicate value
- Order of tuple can have a different sequence

Integrity Constraints

- Integrity constraints are a set of rules. It is used to maintain the quality of information.
- Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.

- Thus, integrity constraint is used to guard against accidental damage to the database.

Types of Integrity Constraint



1. Domain constraints

- Domain constraints can be defined as the definition of a valid set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

2. Entity integrity constraints

- The entity integrity constraint states that primary key value can't be null.
- This is because primary key value is used to identify individual rows and if primary key has null value, then we can't identify those rows.
- A table can contain null value other than primary key field.

Example:

EMPLOYEE

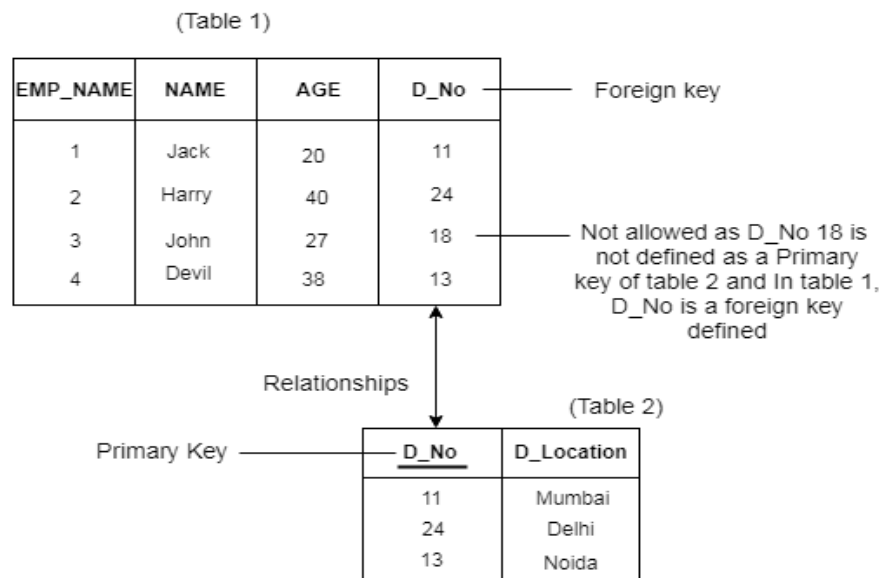
EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

3. Referential Integrity Constraints

- A referential integrity constraint is specified between two tables.

- In Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.



4. Key constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1002	Morgan	8 th	22

Not allowed. Because all row must be unique

Cardinality

Cardinality means how entities are arranged to each other. In DBMS, Cardinality represents a number that denotes how many times an entity is participating with another entity in a relationship.

Cardinality Ratio

Cardinality ratio is also called **Cardinality Mapping**, which represents the mapping of one entity set to another entity set in a relationship set.

For example, if we want to track purchase history of each customer then we can use one-to-many cardinality to find data of a specific customer.

There are four types of Cardinality Mapping:

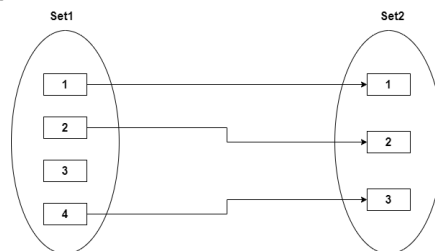
1. One to one
2. Many to one
3. One to many
4. Many to many

1. One to One

One to one cardinality is represented by **1:1** symbol. In this, there is at most one relationship from one entity to another entity.

For example one student can have only one student id and one student id can belong to only one student. So mapping between student and student id will be one to one cardinality mapping.

Another example relationship between director of school and the school because one school can have maximum of one director and one director can belong to only one school.

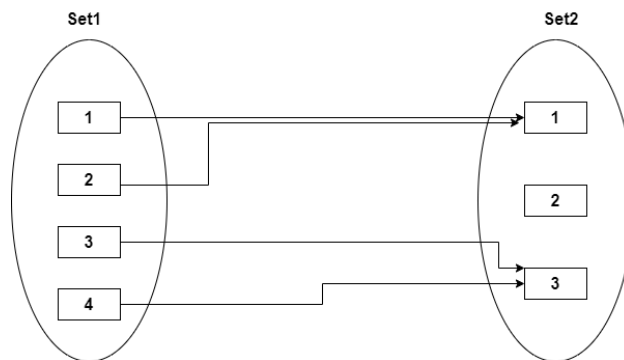


2. Many to One Cardinality:

In many to one cardinality mapping, from set 1, there can be multiple sets that can make relationships with a single entity of set 2. It is represented by **M:1**.

For example, there are multiple patients in a hospital who are served by a single doctor, so the

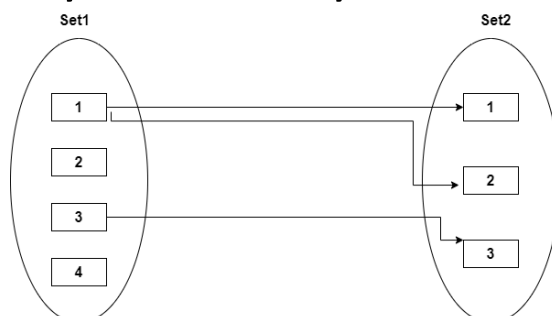
relationship between patients and doctors can be represented by Many to one Cardinality.



3.One to Many Cardinalities:

In One-to-many mapping, from set 1, there can be a maximum single set that can make relationships with a single or more than one entity of set 2. It can be represented by **1: M**.

For Example, in a hospital, there can be various compounders, so the relationship between the hospital and compounders can be mapped through One-to-many Cardinality.



4.Many to Many Cardinalities:

In many, many cardinalities mapping, there can be one or more than one entity that can associate with one or more than one entity of set 2.

It is represented by **M: N** or **N: M**.

One to one cardinality, One to many cardinalities, and Many to one cardinality is the subset of the many to many cardinalities.

For Example, in a college, multiple students can work on a single project, and a single student can also work on multiple projects. So, the relationship between the project and the student can be represented by many to many cardinalities.

